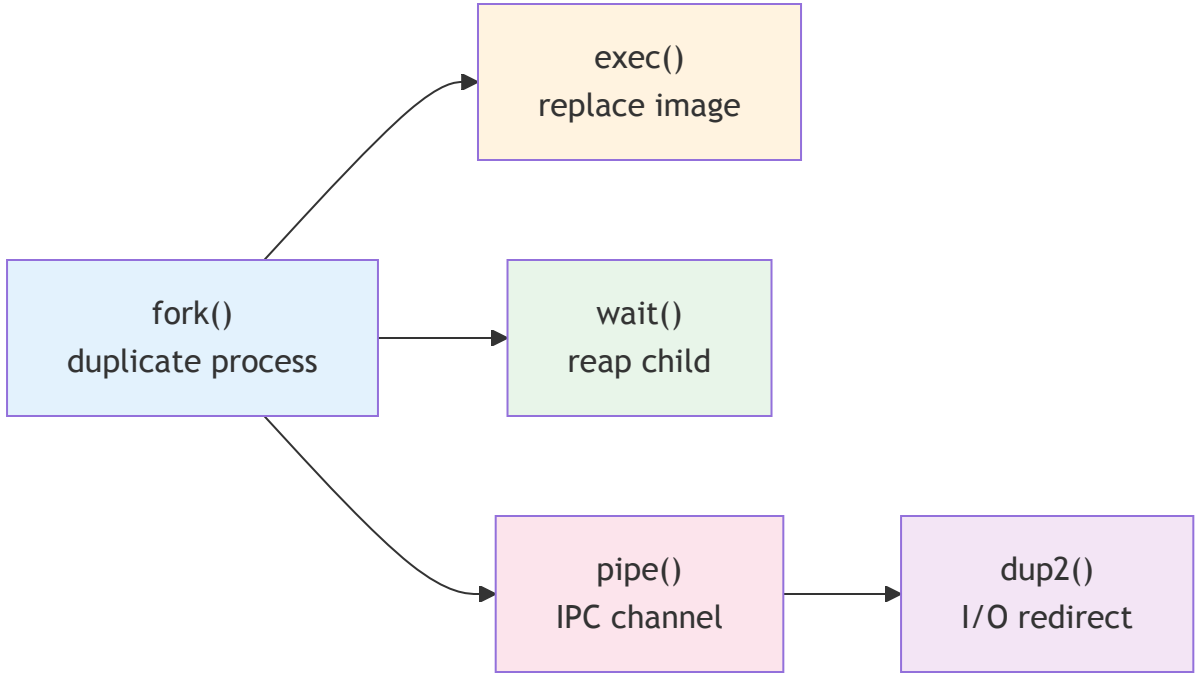
Operating Systems Lab

Week 2 – Process System Calls

Korea University Sejong Campus, Department of Computer Science & Software

Lab Overview

- **Goal**: Practice core UNIX process system calls through C programs
- **Duration**: ~50 minutes · 4 labs
- **Topics**: `fork()`, `exec()`, `wait()`, `pipe()`, `dup()`



Lab 1: fork() and wait() – Problem

Task

Write a program that demonstrates process creation with `fork()` and child process management with `wait()`.

Requirements:

1. Call `fork()` to create a child process
2. In the **child**: print its own PID and parent's PID, simulate work with `sleep(1)`, then exit with code `42`
3. In the **parent**: wait for the child with `wait(&status)`, then retrieve and print the child's exit code using `WIFEXITED` / `WEXITSTATUS`
4. **Bonus**: Create 3 children in a loop, each exiting at different times – observe the termination order

Think about:

- What does `fork()` return to the parent vs. the child?
- What happens if you don't call `wait()`?
- What happens if a child doesn't call `exit()`?

Lab 1: fork() and wait() – Solution

Lab 2: exec() – Problem

Task

Write a program that demonstrates how `exec()` replaces the current process image with a new program.

Requirements:

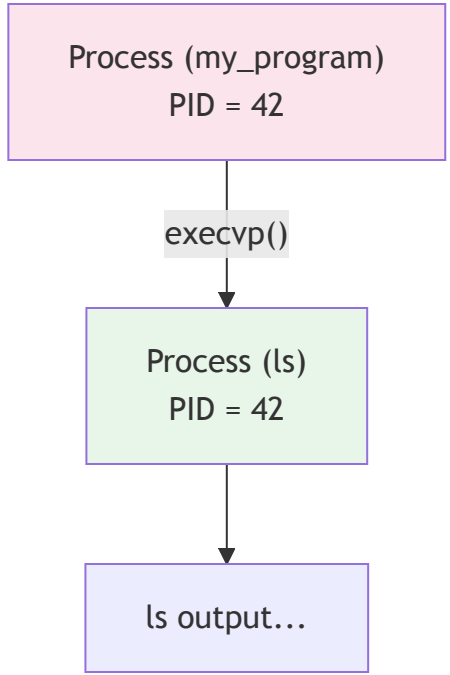
1. Use `execl()` to run `/bin/echo` with a full path
2. Use `execlp()` to run `ls -l -h /tmp` using PATH search
3. Use `execvp()` to run a command with an argument array
4. Demonstrate the **fork() + exec() + wait()** pattern
5. Show what happens when `exec()` **fails** (nonexistent program)

Think about:

- Why does code after a successful `exec()` never run?
- What is the difference between `l` (list) vs `v` (vector) vs `p` (PATH)?
- Why should `exec()` always be called in a child, not the parent?

Lab 2: exec() – Solution

```c
// fork + exec + wait pattern
pid_t pid = fork();
if (pid == 0) {
    // Child: replace with new program
    char *args[] = {"ls", "-l", NULL};
    execvp("ls", args);
    // Only reached if exec fails
    perror("exec failed");
    exit(127);
} else {
    // Parent: wait for child
    int status;
    waitpid(pid, &status, 0);
}
```



exec() family naming:

Suffix	Meaning
`l`	Arguments as a list

Key points:

Lab 3: pipe() – Problem

Task

Write a program that demonstrates inter-process communication using `pipe()`.

Requirements:

1. **Parent → Child**: Parent sends a string message through a pipe, child reads and prints it
2. **Bidirectional**: Use 2 pipes for two-way communication (parent sends question, child sends reply)
3. **Command pipeline**: Implement `echo hello world | wc -w` using pipe + fork + dup2 + exec

Think about:

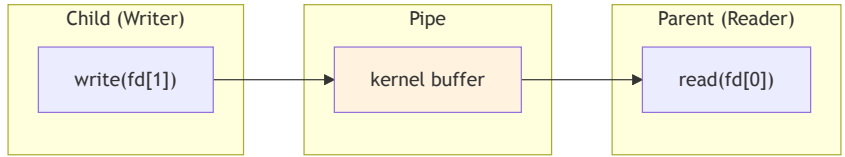
- Why must `pipe()` be called **before** `fork()`?
- Why must unused pipe ends always be `close()`d?
- What happens if you don't close the write end – will `read()` ever return 0 (EOF)?

📁 `examples/lab3_pipe_example.c`

Lab 3: pipe() – Solution

```c
int fd[2];
pipe(fd);  // fd[0]=read, fd[1]=write

if (fork() == 0) {
    close(fd[0]);  // close unused read end
    char *msg = "hello from child";
    write(fd[1], msg, strlen(msg));
    close(fd[1]);
    exit(0);
} else {
    close(fd[1]);  // close unused write end
    char buf[64] = {0};
    read(fd[0], buf, sizeof(buf));
    printf("Received: %s\n", buf);
    close(fd[0]);
    wait(NULL);
}
```



Key rules:

- `pipe()` before `fork()` – both processes share the same fds
- **Always close unused ends** – if write end stays open, `read()` blocks forever (no EOF)
- Pipe is **unidirectional** – need 2 pipes for two-way communication
- If buffer is full, `write()` blocks until reader consumes data

Lab 4: dup2() and I/O Redirection – Problem

Task

Write a program that demonstrates I/O redirection using `dup2()`.

Requirements:

1. **Output redirection** (`>`): Redirect stdout to a file, then run `echo` – verify output goes to the file
2. **Input redirection** (`<`): Create a file with 5 lines, redirect stdin from it, then run `wc -l`
3. **Combined** (`sort < input > output`): Redirect both stdin and stdout, then run `sort`
4. **Bonus**: Demonstrate `dup()` and show that both the original and duplicated fd share the same file offset

Think about:

- Why must `dup2()` be called **before** `exec()`?
- Why close the original fd after `dup2()`?
- How does the fd table survive across `exec()`?

📁 `examples/lab4_redirect.c`

Lab 4: dup2() and I/O Redirection – Solution

```
// Output redirection: cmd > file
int fd = open("output.txt",
    O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fd, STDOUT_FILENO);  // stdout → file
close(fd);                 // close original
execlp("echo", "echo", "Hello!", NULL);
```

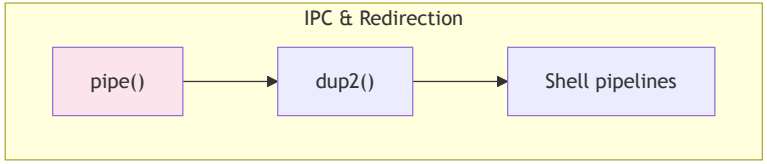
```
// Input redirection: cmd < file
int fd = open("input.txt", O_RDONLY);
dup2(fd, STDIN_FILENO);   // stdin ← file
close(fd);
execlp("wc", "wc", "-l", NULL);
```

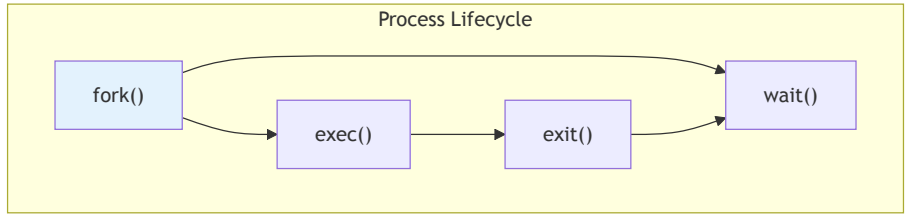
Redirection summary:

Shell	System calls
`> file`	`open(O_WRONLY|O_CREAT|O_TRUNC)` + `dup2(fd, 1)`

How `ls > file.txt` works in a shell:

Key Takeaways





Syscall	Purpose
`fork()`	Create child process (copy of parent)
`exec()`	Replace process image with new program
`wait()`	Block until child exits; reap zombie
`pipe()`	Create one-way IPC channel
`dup2()`	Redirect file descriptors

Shell command `cmd1 | cmd2` → fork + pipe + dup2 + exec(x2) + wait